

Model-Free Passive Execution via Order-Level Shadowing

Vincent Maciejewski
M2 Technologies
mayeski@gmail.com

September 2026

Abstract

Automated execution algorithms are organized into schedule-based and liquidity-seeking families [10, 11]. This paper concerns the first, whose members — Time-Weighted Average Price (TWAP), Volume-Weighted Average Price (VWAP), Percentage of Volume (POV) and Implementation Shortfall — are all *model-based*: each derives its decisions from an explicit model, forecast, schedule or control rule. We introduce SHADOW-PPOV, a passive POV whose *order-placement rate* is set from observed order flow rather than from traded volume. Placing a passive order to fill efficiently conventionally involves an order-book model and a fill prediction. SHADOW-PPOV replaces that prediction with *tracking*: on observing a third-party add, it may transmit its own limit order at the same price *on the same venue*, recording a single association between the observed order’s exchange identifier and its own. Cancellation is then *identifier-driven* — the shadow is withdrawn when the order it follows ends, at once on a cancel and after a brief grace window on a trade. The placement decision is thus *model-free*: price and venue are read off the observed order.

Model-free is not information-free. SHADOW-PPOV reads every order-book message and places only where a participant has just committed capital, while *computing* nothing from what it reads. Information is inherited from the flow rather than derived from a model.

We evaluate SHADOW-PPOV on a full calendar year of replayed Chicago Mercantile Exchange (CME) ES futures in a deterministic market-replay simulator, reporting its slippage and latency sensitivity and comparing it against the aggressive equivalent POV. We propose it as a model-free benchmark for passive-order placement, against which a predictive placement model can be scored. The algorithm and the order-book simulator are implemented in the open-source KASPAR-HFT project.¹

Keywords: optimal execution, percentage-of-volume, adverse selection, limit order book, smart order routing, market microstructure, cancel mirroring.

1 Introduction

A large order cannot be executed in a single trade without paying an impact cost proportional to how much liquidity it consumes at once. The standard response is to slice the parent order into child orders spread over time. The question every execution algorithm must answer is *how to slice*: how much to trade, when, and at what price.

TWAP commits to a clock: it slices evenly in time, blind to where liquidity actually is. But it must decide how much of each slice to work passively rather than cross the spread, and that tactical

¹<https://github.com/vincent212/kaspar-hft>

choice is shared across schedule-based algorithms rather than tied to any one of them [20, 21]. VWAP commits to a *forecast of the intraday volume curve* and weights its slices to match it; the forecast error is the tracking error. Percentage-of-volume (POV) has a participation ratio *reactive* to realized volume — it does not forecast a volume curve the way VWAP does [16, 6]. But POV cannot participate *passively* by reaction alone, because volume that has already printed can no longer be joined at a resting price; to stay passive it must lean slightly ahead of near-term volume (or cross the spread to catch up [13, 18]), and its completion time depends on volume it cannot yet see [17]. So POV trades a volume-*curve* forecast for a short-horizon anticipation plus completion uncertainty, and it tracks an *aggregate* volume statistic rather than the individual resting orders that make up the book.

Implementation Shortfall takes a different benchmark: the arrival price, the market price at the moment the parent order is released. It measures execution cost as the gap between that decision price and the realized average fill price, including the opportunity cost of any unfilled quantity. Minimizing the gap trades market impact, which favors slow trading, against timing risk from price drift and volatility, which favors fast trading. The resulting schedule is front-loaded and tuned by a risk-aversion parameter: higher risk aversion trades faster to reduce exposure to adverse drift, at the cost of more impact. Like the volume-based algorithms, IS trades to a schedule computed in advance from a volume and impact forecast. Perold [12] introduces the IS formulation, and Almgren–Chriss [1] give the arrival-price optimal-execution formulation IS approximates.

This work concerns *passive* execution specifically, so the way POV places passive orders is central. A conventional POV algorithm is a hybrid: it rests passive limit orders when the market allows, but to hold its participation rate it also sends marketable orders, and it crosses the spread to catch up when it falls behind or when a slice deadline arrives [13, 18]. The passive-versus-aggressive choice is not special to POV: schedule-based implementations commonly face the additional tactical question of how much of each slice to work passively rather than cross, and this tactical layer is shared across schedules rather than tied to any one algorithm [20, 21]. A schedule that crossed on every slice would pay the spread needlessly, so even TWAP typically posts each slice passively and crosses only the unfilled residual near the slice deadline [22].

Wherever a slice is worked passively, the same placement problem arises: how far into the book to quote, trading fill probability against market impact and adverse selection [14, 15, 27, 19]. A passive order can simply be posted at the touch, which needs no model; but posting blindly at the touch surrenders control of that trade-off and is prone to adverse selection — the order fills exactly when the level is thinning ahead of an adverse move. The conventional solutions instead build an order-book model and predict where a passive order will fill: optimal placement of limit versus market orders [13], optimal liquidation with limit orders under a fill-intensity model [15], and convex optimization of order placement across the book [18]. SHADOW-PPOV removes the need for an explicit order-book model and fill prediction from the placement decision: it resolves the placement problem by *following*, resting each passive order at the price of an order that just arrived at the book. This replaces order-book modeling and fill prediction with tracking of where liquidity actually rests. SHADOW-PPOV governs the placement of passive orders only; how much of a parent order to work passively versus aggressively is a separate scheduling decision. Production implementations may mix passive and aggressive orders, and this paper makes no claim that aggressive orders are inferior. We compare passive Shadow-PPOV to aggressive POV in Section 8.3.

Model-free is not information-free. The two are easily conflated and the difference decides what this method is. SHADOW-PPOV is saturated in market information: it consumes every add, cancel and trade message on its side of the book, and it acts only where another participant has

just committed capital at a price. A resting limit order is somebody’s conclusion, arrived at by whatever means, that this price is worth standing at. Shadowing it inherits that conclusion. The whole of SHADOW-PPOV’s own contribution is to copy that price and watch that order’s identifier. Everything a placement model would normally supply — a picture of the book, a fill probability, a value for queue position, a view on where the price is going — is simply absent from the decision. It free-rides on the modelling other participants have already done rather than performing any of its own, which is why it can be specified in a page (Algorithm 1) and why its cancellation decision needs no state beyond an identifier.

This also bounds what the method can be expected to do. Inheriting a *placement* judgement is not the same as holding a *directional* view: knowing where a fill is likely says nothing about where the price is going next. Section 10 argues that a passive method without a directional forecast should execute slightly worse than the mid-price whatever else it knows, and Section 8 measures exactly that. The latency results of Section 8.4 then show the inherited information is real and perishable: its value decays as the delay between observing an order and acting on it grows.

Its model-free character sets SHADOW-PPOV apart from standard execution algorithms, and lets it serve as a baseline against which predictive placement models can be measured (Section 10).

Beyond the question of how much to post and when, conventional execution stacks carry two further sources of overhead that this work targets directly:

- **Routing computation.** Smart order routing (SOR) across venues is normally a calculation: aggregate per-venue book depth, compute venue attractiveness scores, model per-venue latency, price maker/taker fees, and optimize a routing objective.
- **State-dependent cancellation.** Order-management systems typically decide whether to pull a resting order by continuously evaluating market state — spread, queue position, fill probability, price competitiveness — which requires maintaining substantial per-order state and adds decision latency.

The premise of this paper is that all three — the forward placement of passive orders, the routing computation, and the state-dependent cancellation — are avoidable for a passive execution mandate, and that avoiding them yields a simple, computationally light method. The algorithm does not need to predict anything; it only has to follow the market. SHADOW-PPOV replaces the forward commitment with an observation: the passive orders other participants have *already committed to the book*. In the price-time-priority (FIFO) model used here, a limit order placed at the same price and venue as a resting participant order joins behind that order in the queue; a sweep must therefore consume the participant order and the depth ahead of the shadow before reaching the shadow. The method replaces the routing calculation with a pass-through of the observed order’s originating venue, and replaces state-dependent cancellation with an identifier rule: the shadow is withdrawn when the order it follows ends.

A note on the word “shadow”. Here it means *attached to*. A shadow order is an ordinary displayed limit order, bound by identifier to a particular third-party order and following that order’s price, venue and lifecycle — it shadows the way a person shadows another, by going where they go. Some venues also use “shadow” as a label for hidden or reserve order types, which is a different word doing a different job; we flag it only so the reader reads ours in the right sense. Every order SHADOW-PPOV sends is displayed.

Contributions.

1. We propose SHADOW-PPOV as a simple, *model-free* benchmark for passive-order placement, built on *participant shadowing*: on observing a third-party add, SHADOW-PPOV places a passive order at that add’s price and venue, attaches it to the arriving order by exchange identifier, and ties its lifecycle to that order. Placement is model-free in the sense used throughout: price and venue are read off the observed order, giving predictive placement models a reference to beat (Sections 6, 10).
2. We define *identifier-driven cancel mirroring*: cancellation triggered by a delete/trade for the followed order’s identifier rather than by market-state evaluation, retaining a single identifier association per followed order — the exchange order identifier paired with the shadow’s — so that no book state is available to the cancellation decision even in principle (Sections 6.5–6.6).
3. We *propose* — and argue analytically, without evaluating — *flow-inherited venue selection*: placing on the originating venue of each shadowed add makes the execution venue distribution match the observed add distribution without any venue score or routing optimizer, an inherited split rather than an optimized one (Section 6.7). The evaluation corpus is a single venue, so this mechanism is unexercised by the measurements reported here; testing it requires a fragmented multi-venue market (Section 11).
4. We describe the *reference implementation* in the KASPAR-HFT system, which layers state-based safety guards (a two-band place/cancel hysteresis plus completion and over-hedge guards) atop the core method, distinguishing the minimal core from the implemented superset (Section 7).
5. We report a measured evaluation on a full calendar year of ES futures, run through the KASPAR-HFT deterministic replay simulator, in which the execution style and the simulated latency are varied one at a time and every other parameter is held fixed (Section 8).

2 Prior art: existing execution methods

Automated execution in current practice is organized around two broad families: schedule-based methods and liquidity-seeking methods [10, 6, 26]. We describe each operationally and note how SHADOW-PPOV differs; the academic results underpinning these methods are reviewed separately in Section 3, and the closest patented mechanisms in Section 4.

Scheduled slicing (TWAP, VWAP). The most widely deployed methods slice a parent order onto a schedule. TWAP spreads child orders evenly in time. VWAP shapes the schedule to a forecast of the intraday volume curve so that the realized average price tracks the volume-weighted benchmark. Both commit, before trading, to a trajectory keyed to a clock or a predicted volume profile, and both must post ahead of where volume will be in order to participate. SHADOW-PPOV commits to no such trajectory: it places only in reaction to orders already resting in the book, and does not forecast or pre-position against future volume. Kissell [6] covers both TWAP and VWAP in detail.

Percentage-of-volume (POV). A POV algorithm holds a target participation rate in market volume, scaling its trading up and down with *realized* volume rather than a fixed clock. It does not forecast a volume curve, but it does track an *aggregate* statistic (total traded volume) and, to participate passively, must lean slightly ahead of near-term volume rather than cross the spread. SHADOW-PPOV is POV-like in effect — its trading scales with the rate of observed adds — but its

participation emerges from following *individual* resting orders at a bounded rate, and it does not pre-position against future volume. Kissell [6] covers POV in detail.

The POV control loop, and how Shadow-PPOV differs. A conventional POV algorithm runs a feedback loop against an aggregate statistic. Let ρ be the target participation rate, V_t the cumulative market volume, and x_t the quantity executed so far. At each step the algorithm compares x_t against the target ρV_t : when $x_t < \rho V_t$ it trades more, posting aggressively or crossing the spread to catch up; when $x_t > \rho V_t$ it slows or pauses.

SHADOW-PPOV replaces this control loop with per-order following (Section 6). Its actions are driven by the arrival of individual orders: on each observed add it may post a shadow at that order’s price and venue, bounded by that order’s size. The shadow rests just behind a live order at the same price, so under FIFO priority a sweep must consume the followed order and the depth ahead of the shadow before reaching it. Participation emerges from the rate of order arrivals, placement and venue selection are inherited from the followed order, and a forecast of forward volume is unnecessary. Table 1 contrasts the two mechanically.

Table 1: Conventional POV and SHADOW-PPOV, by mechanism.

Aspect	Conventional POV	SHADOW-PPOV
Quantity tracked	aggregate market volume V_t	individual resting orders
Participation	target rate ρ , held by a feedback loop	emergent from following adds at a bounded rate
Passive placement	leans ahead of forecast volume	mirrors orders already resting
When behind	crosses the spread, paying impact	following continues until the target size is filled
Book / venue placement	separate SOR and queue logic	inherited from the followed order
Fill rate	depends on placement quality	requires a sweep that clears the followed order first (shadow rests just behind)

Smart order routing (SOR). When liquidity is fragmented across venues, SOR decides where to send each child order by computing and comparing per-venue liquidity, fees and rebates [25], fill rates, and latency, and optimizing a routing objective [24, 26, 18]. SHADOW-PPOV performs none of this: it sends each shadow to the originating venue of the add it follows, so the venue mix is inherited from observed order flow (Section 6.7) rather than solved for.

Liquidity-seeking algorithms. A liquidity-seeking algorithm aims to complete an order by capturing whatever liquidity is available at acceptable prices, prioritizing completion and low information leakage over any schedule or benchmark. As the second family of execution methods, it abandons a fixed schedule and sources liquidity opportunistically: posting and taking across lit and dark venues, and using hidden and iceberg orders and immediate-or-cancel probes to find undisplayed block size [6, 26]. It raises participation when liquidity appears at an acceptable price and withdraws otherwise, and may finish early when a block is found or wait when nothing suitable appears. SHADOW-PPOV operates on CLOB venues only, following the *displayed* resting orders on the book. It does not route to dark pools and makes no attempt to detect hidden or iceberg liquidity

— it follows what the book shows, and commits to no trajectory of its own. SHADOW-PPOV and a liquidity seeker pursue different goals: a liquidity seeker minimizes footprint and hunts for size across lit and dark venues; SHADOW-PPOV places a passive order well on the lit book once a schedule has already decided to rest one. The two families are complementary — a liquidity seeker decides *when* enough liquidity is present to act, whereas SHADOW-PPOV decides *where* to rest a passive order.

Quote matching. The closest prior strategy in spirit is quote matching, described by Harris [5]: a trader observes a large standing limit order and places an order one tick *better*, taking priority ahead of it. The standing order is thereby converted into a free option — if the price moves against the quote matcher, it exits by trading *against* that order, which is the very liquidity it stepped in front of. This shares one element with SHADOW-PPOV: both act on an *individual* identified resting order rather than on aggregate book state. Everything else is opposed.

SHADOW-PPOV places at the *same* price, joining behind the observed order in the queue rather than ahead of it, so it takes no priority the other participant would otherwise have held. It shadows at a fixed rate irrespective of size, rather than selecting large orders for the size of the option they offer. It never trades against the order it follows: when that order ends, the shadow is withdrawn (Section 6.6). And its purpose is to complete a parent order at a predictable cost, not to extract an option from another participant’s commitment.

The comparison is also where the cancellation mechanism comes from. Harris’s stated objection to quote matching is that the large standing order can simply vanish, leaving the quote matcher exposed precisely when the price then moves against it. SHADOW-PPOV’s exposure to that event is the same in kind, and identifier-driven cancel mirroring is the direct answer to it: the shadow is pulled on the delete or trade message for the specific order it follows, without evaluating any market state (Sections 6.5–6.6). The failure mode that makes quote matching hazardous is the event this method is built to react to.

3 Related literature

The methods above rest on, and are analyzed by, a substantial academic literature; we review the strands most relevant to SHADOW-PPOV’s design choices.

Optimal-execution theory. The modern foundation is Almgren–Chriss [1], which casts execution as minimizing a mean–variance trade-off between market impact (favoring slow trading) and timing/volatility risk (favoring fast trading), yielding an optimal schedule given models of permanent and temporary impact. A large body of work refines this framework, including limit-order formulations with adverse selection and fill uncertainty in the spirit of Avellaneda–Stoikov [2]. This theory answers *how fast* to trade given an impact model and a volume profile; it presupposes the profile. SHADOW-PPOV keeps the passivity these analyses target but does not compute a trajectory — it follows one the market reveals.

Queue position and adverse selection. Under price–time priority, both fill probability and adverse-selection cost depend on where a resting order sits in the first-in, first-out (FIFO) queue. Moallemi–Yuan [23] give a valuation model in which adverse-selection cost *increases* with queue position, with a dynamic component capturing the option value of locking in a position early. This literature quantifies precisely the effect SHADOW-PPOV is designed around: leading the book — posting early and deep in a thin queue — buys priority but courts selection when the level thins ahead of an adverse print. By posting alongside an order a participant has already chosen to rest,

SHADOW-PPOV avoids deliberately taking priority ahead of that order. The followed order therefore provides an observable reference for the local queue environment without requiring SHADOW-PPOV to predict that environment.

Order-flow competition and routing. Foucault–Menkveld [24] analyze competition for order flow across venues and the routing systems that arbitrate it, and Laruelle–Lehalle–Pagès [26] give stochastic-approximation algorithms that learn the optimal split of an order across liquidity pools. Where this literature *solves* for the venue split, SHADOW-PPOV’s split *emerges* from placing on the originating venue of each followed order (Section 6.7).

Practitioner references. The practical construction and calibration of VWAP/TWAP/POV and their trade-offs are documented in practitioner texts such as Kissell [6], which we take as the reference description of the methods in Section 2.

Recent execution research. Contemporary execution research is largely forecast- and optimization-based, which sharpens the contrast with SHADOW-PPOV. Reinforcement-learning formulations place market and limit orders to minimize implementation shortfall, both in general limit-order settings [29] and within queue-reactive order-book models [28], and signal-adaptive schemes optimize quote placement from predictive features. Optimal execution under *passive* market impact [27] models fill and impact as functions of spread, queue position, displayed depth, imbalance, and recent order flow — the same state SHADOW-PPOV does not compute. Work on the market maker’s trade-off between fill probability and post-fill return [30] quantifies the top-of-book adverse selection that any resting order, including a shadow, inherits (Section 10). SHADOW-PPOV differs from all of these in computing nothing at all: it places in reaction to observed resting orders.

4 Prior patents

Four patented mechanisms sit closest to SHADOW-PPOV. Each shares a surface resemblance — passivity, pegging, or following — but differs in a way that is material to SHADOW-PPOV’s design.

- **Pegged / tracking orders** (e.g. US 10,614,520, “Tracking Liquidity Order” [33]). A non-displayed order pegs to the same side of the National Best Bid and Offer (NBBO), continuously *re-pricing against evaluated market state*. SHADOW-PPOV evaluates no such state to manage its shadow: price is inherited once from the shadowed add, and the shadow is *cancelled* when that add ends.
- **Copy / mirror trading** (e.g. US 2013/0268423, “Copy Trading System and Method” [34]). These replicate *executed trades* of *identified* accounts a user has chosen to follow. SHADOW-PPOV follows *anonymous passive adds* at the message level — the posting of liquidity itself — and couples both entry *and cancellation* to the followed order’s identifier.
- **Exchange-side passive/hidden liquidity** (e.g. US 2006/0253379, “Passive Liquidity Order” [35]). This is a matching-engine mechanism internal to a venue; SHADOW-PPOV instead is a client-side strategy that observes a public feed and places and cancels its own orders.
- **Hidden-liquidity detection** (e.g. US 8,140,416 [36]). Here order identifiers are tracked for *analysis* (inferring hidden size), and trading decisions still evaluate market state; SHADOW-PPOV uses identifier presence directly as the trigger for a cancellation.

5 Positioning relative to prior work

Having reviewed the existing execution methods (Section 2), the academic literature (Section 3), and the closest patented mechanisms (Section 4), we state where SHADOW-PPOV sits relative to all three. The contribution is not the general idea of following displayed liquidity or using participation rates, which are established in both the public literature and prior patents. It is the specific combination of order-level shadowing, identifier-keyed lifecycle mirroring, and same-venue placement as a model-free passive-placement benchmark — a combination of design choices that all follow from one commitment, to build no model of the book and follow observed orders instead:

1. **Model-free placement.** The passive order’s price is inherited from the observed add it follows, so placement needs no order-book model and no fill-probability prediction; this is what lets SHADOW-PPOV serve as a benchmark for predictive placement models (Section 10).
2. **Identifier-keyed cancellation.** A shadow-order is withdrawn when a delete or trade message for the specific order it follows arrives, rather than when evaluated market state (spread, queue decay, fill probability) crosses a threshold.
3. **Flow-inherited venue selection.** Each shadow is sent to the originating venue of the add it follows, so venue choice is a pass-through rather than an optimization; the cross-venue mix of executions thereby tracks, in expectation, the cross-venue mix of observed liquidity provision without explicit routing logic — the inverse of the SOR approach [24, 26], where the split is solved for.

Table 2 summarizes the positioning.

Table 2: How SHADOW-PPOV differs from the nearest prior work.

Prior work	Mechanism	How SHADOW-PPOV differs
Almgren–Chriss [1]	Optimal <i>schedule</i> from a volume/impact model	No schedule; participation emerges from following individual adds
POV [6]	Reactive participation to <i>aggregate</i> realized volume; leans ahead to stay passive	Follows <i>individual</i> resting orders; never posts ahead
Moallemi–Yuan queue value [23]	Quantifies queue-position adverse selection	Does not lead the queue; inherits a resting order’s neighborhood
SOR [24, 26]	Computes or learns the venue split	Split <i>emerges</i> from same-venue placement; no routing computed
Tracking/pegged orders [33]	Re-prices against evaluated NBBO state	No state evaluated; cancels on followed-order termination
Copy/mirror trading [34]	Copies executed trades of identified accounts	Follows anonymous passive <i>adds</i> ; keys cancellation to their identifier

6 The Shadow-PPOV Method

6.1 Message model

SHADOW-PPOV consumes a real-time stream of order-book messages from one or more venues (e.g. an ITCH-style binary feed). Each message carries a *message type* — ADD (a new resting limit order),

DELETE (cancellation), TRADE (execution), MODIFY (amendment) — an *exchange-assigned order identifier*, and an *originating venue identifier*, together with side/price/quantity for ADD/MODIFY. Individual order lifecycles are therefore followable at the message level by identifier.

6.2 Core mechanic: participant shadowing by identifier

On each ADD that satisfies a selection criterion (below), SHADOW-PPOV transmits a *shadow* limit order at the add’s price to the add’s originating venue, and receives a venue-assigned shadow-order identifier. It then records a single association keyed by the observed order’s (venue, `exchange_order_id`) — exchange order identifiers are unique only within a venue — and mapping it to its shadow-order identifier,

$$(\text{venue}, \text{exchange_order_id}) \mapsto \text{shadow_order_id}.$$

The shadow is thereafter *bound* to the observed order by that key: it inherits that order’s price and venue, and — crucially — its *lifecycle*. The single invariant that makes this “shadowing” rather than “quoting” is this attachment; a shadow is a projection of one identified resting order.

6.3 Order-placement rate

SHADOW-PPOV shadows a bounded fraction of qualifying adds. The reference implementation offers two independent selection modes:

- *Probabilistic*: shadow a qualifying add with a fixed probability ρ , drawn independently per light. ρ is a free parameter; the values used here are stated with each experiment.
- *Deterministic*: shadow every N -th qualifying add.

The two modes give different selection rates. The experiments reported in Section 8 use the probabilistic mode. The quantity SHADOW-PPOV sets is thus the *order-placement rate* ρ — the fraction of qualifying adds it shadows — and *not* a participation rate. This placement rate is fixed by construction and applied to observed order flow, not to traded volume; SHADOW-PPOV places more shadows when adds are frequent and fewer when they are sparse. The realized *participation* — SHADOW-PPOV’s share of traded volume — cannot be controlled directly; it emerges from following at the fixed placement rate. Because not every placed shadow fills, the realized participation is not equal to the order-placement rate and will generally differ from it. Participation is therefore a random quantity whose dependence on parent-order size is analyzed in Section 9.

6.4 Shadow quantity

The shadow’s *desired* quantity is computed independently of the triggering add’s quantity — in the reference implementation as a randomized function of average trade sizes over a preceding window. Before transmission the desired quantity is capped by the followed order’s size, the per-order and per-level limits (e.g. 1-lot child orders with a 25-lot aggregate ceiling), and the remaining parent quantity. Fills follow from co-resting at the same price: a sufficiently large sweep can consume the followed order and continue into the shadow, while a partial sweep may stop before reaching the shadow.

6.5 Retained information

For each followed order, the core association retains *only* its (venue, exchange identifier) key paired with the shadow identifier; it does not retain the followed order’s price, quantity, queue position,

order-book state, or timestamp. The algorithm additionally maintains global configuration — target side, selection rate, and grace-window parameters — together with the minimal state needed to manage its own orders. Two consequences follow. First, no price/queue/state-based cancellation decision is *possible*, because the inputs to such a decision are not retained; cancellation is forced to be identifier-driven by construction. Second, storing, looking up, and removing an association by identifier is expected $O(1)$, so the cost of the cancellation decision is independent of the number of tracked orders.

6.6 Identifier-driven cancel mirroring

On a lifecycle-terminating message for a followed order, SHADOW-PPOV looks up the message’s exchange identifier. If no association is present, nothing happens. If it is present, the response depends on *how* the followed order ended:

- **Pulled (delete).** The liquidity simply left; no aggression is in flight to fill against. SHADOW-PPOV cancels the shadow *immediately* and removes the association — continuing to rest there would be pure adverse-selection exposure.
- **Hit (trade).** A trade is a marketable sweep still in flight, and because the shadow rests in the same queue behind the followed order, that sweep may be about to reach it. Cancelling instantly would pull the shadow out of the queue *before* the aggression arrives — and in the worst case the shadow would then *never* fill, because it always cancels in time. SHADOW-PPOV therefore arms a *delayed cancel*: it holds the shadow live for a short grace window of B end-of-burst events, during which the shadow may itself be filled by the continuing sweep. If the window elapses without a fill, the cancel is sent and the association removed. Arming is idempotent: a further trade for the same order while the window is armed does not restart it.

Aside from this trade-versus-pull distinction, the cancellation input is still just identifier presence: no book depth, queue position, price competitiveness, or fill probability is evaluated. The hard safety guards of the reference implementation (Section 7) — target reached, over-hedge, or price drifted past the cancel band — may fire an *immediate* cancel that pre-empts an armed grace window. A shadow therefore never outlives the order it follows by more than the grace window, preserving the discipline that SHADOW-PPOV rests only where a currently-live participant order rests, or rested an instant ago. A MODIFY for a tracked order is treated as termination of the followed lifecycle: the shadow is cancelled and the association removed. Re-shadowing is not automatic — the amended order is shadowed anew only if the feed subsequently emits a fresh qualifying ADD for it. This is an algorithmic convention rather than an assertion about exchange modify semantics.

6.7 Flow-inherited venue selection

Because each shadow is transmitted to the originating venue of the add it follows, a fixed-rate selector preserves, in expectation, the cross-venue distribution of qualifying observed adds. If participants rest 50%/30%/20% of adds on venues A/B/C, a fixed selection rate reproduces that 50:30:20 mix in expectation — *without* venue liquidity metrics, fee/rebate optimization, latency-based routing, or any explicit routing logic. This is flow-inherited venue selection, not an optimization: the venue mix is proportional to the observed distribution of qualifying adds, which need not coincide with the split that minimizes fees, latency, or adverse selection, nor with the distribution of traded volume.

Algorithm 1: SHADOW-PPOV identifier-driven core (per message)

state: mapping $M : k \mapsto \text{shadow_id}$ with venue-scoped key $k = (\text{venue}, \text{ex_id})$ (exchange ids are unique only within a venue); target side σ ; order-placement rate ρ ; grace window B bursts; own-id set O of $(\text{venue}, \text{shadow_id})$ pairs

Input: message m with type $m.\tau$, exchange id $m.\text{id}$, venue $m.v$, side $m.s$, price $m.p$; write $k = (m.v, m.\text{id})$

```
1 switch  $m.\tau$  do
2   case ADD do
3     if  $k \in O$  or  $m.s \neq \sigma$  then return           // self / wrong side
4     if selection criterion not met (prob.  $\rho$  / every- $N$ ) then return
5      $\text{shadow\_id} \leftarrow$  transmit shadow at price  $m.p$  to venue  $m.v$ ;
6      $O \leftarrow O \cup \{(m.v, \text{shadow\_id})\}$ ;  $M[k] \leftarrow \text{shadow\_id}$ 
7   case DELETE (followed order pulled) do
8     if  $k \in M$  then cancel  $M[k]$  now; remove  $M[k]$ 
9   case TRADE (followed order hit) do
10    if  $k \in M$  and not already armed then
11      arm delayed cancel of  $M[k]$  in  $B$  bursts      // grace: the sweep may still
12      fill the shadow
13  case MODIFY do
14    if  $k \in M$  then
15      cancel  $M[k]$  now; remove  $M[k]$                 // teardown

// grace-window expiry or safety guard  $\Rightarrow$  cancel armed shadow; own fill  $\Rightarrow$ 
// remove its association
// on parent complete: cancel all shadows in  $M$ ; clear  $M$  (wind-down)
```

7 Reference Implementation: the kaspar-hft System

KASPAR-HFT is an open-source, low-latency trading system and order book simulator written in C++.

7.1 Simulation facilities

The simulator is the instrument this paper is written with, so it is worth stating what it provides. It reconstructs a full order-by-order (MBO) book from recorded exchange packet captures, maintaining every resting order individually rather than aggregated depth, which is what makes a queue position — and therefore a fill inference — definable at all. Three latencies are configurable and independent: the outbound order, the outbound cancel, and the inbound market-data feed.

The fill inference. A shadow enters the simulated price-time-priority (FIFO) queue at its price, behind the depth already resting there. It is assigned a fill when subsequent recorded executions imply that aggression has reached a resting order *behind* it: the shadow has earlier time priority than that order, so an aggressor that reached the one must have passed through the other.

Market impact. The simulator leaves it out. Orders are filled against the recorded feed as though they had not been there: the book carries on exactly as it did on the day, so a quote the algorithm sits in front of stays where it is and a counterparty that would have repriced does not. Every cost reported here is therefore a cost before the market’s reaction to the order that produced it.

It is built using the actor-model architecture [3, 4]: independent components communicate only by passing messages, so concurrent order handling stays deterministic and keeps shared-memory locking off the critical path. The same code runs unchanged across backtesting, market replay, live simulation, and live trading; the evaluation of Section 8 therefore exercises the live code path directly.

7.2 Passive POV implementation

SHADOW-PPOV is implemented as a set of execution components within KASPAR-HFT. A strategy issues a position order, and the execution components translate it into the individual limit orders sent to the exchange, with the acquiring and liquidating sides of each instrument handled independently. This implementation is a *superset* of Algorithm 1: it keeps the identifier-driven attach/cancel core and adds state-based *safety guards* appropriate to running real capital against a live venue.

The added guards are:

- **Two-band place/cancel hysteresis.** A shadow is only *placed* when its price is within a tight placement band δ_{place} ticks of the inside (e.g. for a buy, $p \geq b_0 - \delta_{\text{place}}$ against best bid b_0), but is only *cancelled for distance* once it drifts beyond a looser cancel band δ_{cancel} , with $\delta_{\text{place}} < \delta_{\text{cancel}}$. The gap introduces hysteresis: it prevents a one-tick oscillation of the inside from triggering a place/cancel/replace loop that would burn message budget and queue priority. The placement band δ_{place} also bounds how far from the inside a shadow can attach; tightening it restricts shadowing to adds near the touch, which fill more reliably and reduce the small-order participation dispersion of Section 9.
- **Completion and over-hedge guards.** A working shadow is also pulled when the position reaches its target or goes flat, when the size resting at its price would overshoot the quantity that remains to be executed, or when per-level order caps are exceeded. The placed size is the minimum of the per-level cap, the per-order cap, the remaining quantity, and the followed order’s size, clamped at zero. Between them the guards hold the shadow within the quantity that remains and within the size of the participant it follows.

Grace window. As described in Section 6.6, the shadow’s cancel is delayed only when the followed order is hit by a trade, to give the incoming sweep a chance to fill the shadow; when the followed order is instead cancelled, the shadow is cancelled at once. The safety guards override this delay: whenever a completion, over-hedge, or distance limit is reached, the shadow is cancelled immediately.

In short, the core method is the minimal identifier-driven algorithm (Algorithm 1); the reference implementation runs that core plus the guards above. Guarded cancellations are a strict addition to the identifier-keyed cancel.

The guards read market state — the inside price and recent trade sizes — and it is worth being precise about what that does and does not mean for the model-free claim. *Model-free* here means what it means throughout the paper: no order-book model and no fill prediction. A band on distance from the touch, a cap on size resting at a price, and a limit on position against target are risk limits. None of them forecasts a fill, values a queue position, or estimates where the price is going, and none of them chooses *where* to place — the price is still inherited from the shadowed

order, and the cancellation is still driven by that order’s identifier. Reading the inside price to enforce a band is not a model of the book any more than a position limit is a model of the market. The guards bound what the method may do; they do not supply it with a view.

7.3 Naive aggressive POV

The aggressive baseline that Section 8 measures against is the same shadowing mechanism driven by the *opposite* signal. Where passive SHADOW-PPOV shadows resting ADDS, the aggressive arm shadows *aggressive* orders — the trade prints. On each observed TRADE, it flips a coin with a fixed per-trade probability (a placement rate in basis points of print) and, when selected, places a marketable order at the print’s price that follows the aggressor into the book. This is a *naive* POV: it participates by taking a fixed fraction of trading activity directly, with no volume-curve forecast and no participation-feedback loop — the same construction as passive SHADOW-PPOV, applied to trades rather than adds. In the reference implementation both arms are in fact the *same* C++ class (Actor).

This arm is standard practice, not a construction of ours. Print following is what commercial POV implementations already do, and we claim no novelty for it. Deltix’s PVOL states the rule explicitly: it divides the order duration into intervals, and “at the end of each interval, PVOL calculates the total volume of trades that happened on the market”, then sizes the child order as $\text{Quantity} = \text{VOL}(T_i) \times \text{ParticipationVolume}$ [31] — observe the prints, then send a marketable child sized as a fraction of them. TradeStation describes its POV route the same way, as an order that “trades based on real time market volumes” and is worked with market or marketable limit orders at algorithm-determined intervals [32]. A POV algorithm is therefore a liquidity *taker* that waits for volume to print and participates afterwards [6].

The aggressive arm here is that rule taken to its limiting grain: instead of aggregating prints over a 30-second interval and crossing once, it acts on each print individually. That is a difference of granularity, not of kind, which is precisely why it makes a fair baseline — the passive method is being compared against what desks actually run, rather than against a strawman built for the purpose. The novelty claimed in this paper is on the *passive* side (Sections 6.5–6.6), where we have found no published method that places at an identified individual order’s price and ties cancellation to that order’s exchange identifier.

Two mechanical details distinguish it from the passive arm. The aggressive order is an immediate-or-cancel in intent: it is given a short time-to-live (default 1 ms) so that a cross which arrives after the level it was priced from has already been consumed is pulled rather than left to rest and silently become a passive order the arm never asked for. This is what is referred to as “Aggressive POV” throughout Section 8.

We carry the aggressive arm because the comparison it enables is the practically important one. The question a desk faces is not whether shadow-following works in the abstract but how passive placement compares to simply crossing, and a practitioner will typically run both versions side by side and choose per order. Implementing the two in one class lets that comparison be made like-for-like — same instrument, corpus, and position mandate — which is exactly the passive-against-aggressive experiment of Section 8.3.

8 Experimental Evaluation

We evaluate the baseline algorithm performance here. Since our simulator does not model market impact, the algorithm’s measured cost carries no component that scales with participation rate or

order size. We therefore focus on the base case, where slippage is measured in idealized conditions. We then compare the passive SHADOW-PPOV against its aggressive equivalent. Finally we check the latency degradation characteristics.

8.1 What the experiment is

Instrument and corpus. E-mini S&P 500 futures (CME product ES), market-by-order data on CME channel 310, every regular trading session of calendar year 2025 for which PCAP files were available.

What the algorithm is asked to do. The algorithm is run as a *two-sided market maker*, quoting the buy and the sell side simultaneously throughout the regular session, and is required to complete a fixed quantity on each side within each measurement window. Concretely:

- **Window boundaries** are placed every 10 minutes from 09:29 ET to 16:00 ET. Each boundary closes one window and opens the next, so a session yields at most 39 windows.
- **At each boundary** the algorithm is given 100 contracts to buy and, independently, 100 contracts to sell. The two are worked over the same window by separate books, so both sides quote at once rather than one after the other.
- **A window closes** when *both* conditions hold: the 10-minute minimum has elapsed, *and* both sides have completed their full 100 contracts. A session in which the algorithm fills slowly therefore produces *fewer and longer* windows, never a partially filled one.

8.2 What is measured

All prices are in ticks; one tick is 0.25 index points, which is \$12.50 per contract on ES.

Relative slippage. Let vwap_{BUY} and vwap_{SEL} be the size-weighted average prices at which the window’s 100 bought and 100 sold contracts were actually executed, and let m_0 be the mid-price at the instant the window opened — the arrival price for both legs, since both start together. Each leg is priced against that mid,

$$\text{Slippage}_{\text{BUY}} = \text{vwap}_{\text{BUY}} - m_0, \quad \text{Slippage}_{\text{SEL}} = m_0 - \text{vwap}_{\text{SEL}}, \quad (1)$$

signed so that a positive value is a loss on either side, and the reported figure is the average of the two:

$$\text{RelativeSlippage} = \frac{1}{2}(\text{Slippage}_{\text{BUY}} + \text{Slippage}_{\text{SEL}}) = \frac{1}{2}(\text{vwap}_{\text{BUY}} - \text{vwap}_{\text{SEL}}). \quad (2)$$

We call this *relative slippage*: relative to the arrival mid, and relative between the legs. The second equality is the reason to average rather than report the legs separately: m_0 cancels, so whatever the market did during the window affects both legs and drops out. The per-leg figures are considerably noisier for exactly that reason — over a leg of a minute or more the market drifts, and the drift lands in the two legs with opposite signs. Measured over the corpus the buy and sell legs carry 95 % intervals about four times wider than their own average. The paired form is immune to that drift by construction, which is what makes a full-year average meaningful.

Both legs start together but do not necessarily finish together.

Quoting both sides at once raises the question of whether the algorithm’s own bid- and offer-side shadows interact. They cannot trade against each other: the simulator matches each shadow against the recorded feed, never against another shadow. Nor can they queue behind one another, since each attaches to a third-party order on its own side of the book. On ES, which is one tick wide at

almost every window opening, there is rarely an intervening price level at which the two sides could meet. The interaction that does remain is through the position and completion guards of Section 7, which see net position and can therefore pull a shadow on one side because of fills on the other.

Cost relative to immediate execution. The alternative to placing passively is to cross the spread at once, priced against the same m_0 : a buy crosses to the ask and a sell to the bid, so immediate execution has a slippage of half the quoted width. Over this corpus the ES book is one tick wide at 96.3 % of window openings, giving a mean spread of 1.038 ticks and an immediate-execution slippage of +0.519 ticks per contract. The difference,

$$\text{RelativeToImmediate} = \text{RelativeSlippage} - \frac{1}{2}(\text{ask}_0 - \text{bid}_0), \quad (3)$$

is negative when the passive execution finished at a better average price than crossing at the open would have. Both quantities in it are slippages against the same arrival mid, so the subtraction is between like things.

Cost relative to VWAP. The same executions compared against the volume-weighted average price of every trade that occurred in the market while the leg was working, rather than against the arrival mid. A negative value means the algorithm executed better than the average price paid by everyone else trading over the same interval. This measure is not comparable across configurations whose legs run for different lengths of time: each is scored against its own interval, and a longer leg averages over a broader and more forgiving slice of the day.

Participation rate. For one side of one window, let F be the contracts the algorithm filled and V the contracts traded by everyone else over that same side’s duration. The participation rate is

$$\text{Participation} = \frac{F}{F + V}, \quad (4)$$

bounded in $[0, 1]$, which is what a percentage-of-volume mandate means by the term. One number is reported per cell: the *aggregate* rate $\sum F / \sum (F + V)$ over every side of every window, which is the share of the year’s volume the algorithm took and the figure a mandate is written against. The mean of per-window ratios is not used — it is dominated by windows with very small denominators and reverses the ordering entirely. Sides alongside which no other trade occurred leave the ratio undefined and are excluded rather than recorded as 100 %.

Both sides of a window are pooled, because the window is a 100-contract buy and a 100-contract sell and the two are symmetric by construction. Measured separately they agree to within 0.08 percentage points in every cell, so pooling costs nothing and reporting one side alone would be an arbitrary choice between two answers that do not differ.

Time to fill. The mean wall-clock time for a side to complete its 100 contracts, pooling both sides. The 100 contracts arrive as roughly 85 separate executions of 1.18 contracts on average, in every cell. The clip size is not a parameter: a shadow’s size is bounded by the size of the order it follows, so it is the market’s own add-size distribution that sets it.

8.3 Results

Passive against aggressive. The informative comparison is between the two execution styles. Both execute the same parent over the same corpus; one rests and one crosses; and they are

Table 3: Passive and aggressive execution at nearly matched participation. ES futures, 246 sessions of calendar 2025, 100 contracts bought and 100 sold in every window, zero simulated latency on all three paths. Cost is in ticks per contract: a positive slippage is a loss, a negative relative cost is a gain. Intervals are 95 % and clustered by session, since windows within a day are not independent. *Crossed* is the share of executed quantity that filled on arrival rather than resting: it is the line that separates the two styles mechanically, and the cost lines agree across it.

	SHADOW-PPOV (passive)	Aggressive POV
Shadowed event	resting add	trade print
Rate	5 % of adds	2 % of prints
Completed windows	9,592	9,581
Relative slippage (ticks/contract)	$+0.0955 \pm 0.0130$	$+0.0952 \pm 0.0135$
of which, buy leg	$+0.0704 \pm 0.0532$	$+0.0292 \pm 0.0830$
of which, sell leg	$+0.1205 \pm 0.0517$	$+0.1613 \pm 0.0836$
Relative to immediate execution	-0.4236 ± 0.0117	-0.4239 ± 0.0105
vs. VWAP	-0.6208 ± 0.0204	-0.4036 ± 0.0275
Participation (%)	4.46	4.86
Time to fill 100 (s)	72.7	62.4
Child executions per leg	85.5	85.6
Crossed (%)	1.04	51.86

configured so that the realized participation is approximately equal, which is the only basis on which their costs can be set side by side. Participation is not a control input in either style: neither can be told to hit a target rate — it emerges from following at a fixed placement rate (Section 6.3) — so it cannot be forced to match. Instead of forcing it, we swept the placement rates and selected the pair of settings whose realized participation came out empirically closest, 4.46 % passive against 4.86 % aggressive.

Table 3 gives the relative slippage: +0.0955 ticks per contract for passive against +0.0952 for aggressive, and relative to immediate execution -0.4236 against -0.4239 — agreement to the fourth decimal on both, over more than nine thousand completed windows each.

This is the prediction of the Glosten–Milgrom account of the spread [14] and is discussed in Section 10: the half-spread a resting order captures is returned, in expectation, through adverse selection, and the half-spread a crossing order pays buys the avoidance of it. A method with no forecast cannot separate the two, so the choice between resting and crossing is a choice about speed and participation, not about price.

The VWAP column is the one that differs. At -0.621 against -0.404 passive SHADOW-PPOV appears to beat the market by half again as much. The difficulty is that each leg is scored against the volume-weighted price over its own working interval, and those intervals are not the same length — in either style the buy leg and the sell leg finish at different times, and passive SHADOW-PPOV’s legs run longer than aggressive POV’s. A longer interval averages over a broader slice of the session. So even where a leg beats VWAP, the margin does not translate into usable alpha, and the column should not be read as a ranking of the two methods.

Post-fill mark-outs cross-check. Table 4 reports the mid-price move over the one, five and thirty seconds after each individual fill, signed so that a positive value is a move against the fill. The mark-outs are close to the relative-slippage measure, which is the result we expect. The two are related rather than identical: one is the price a completed round trip achieved, the other is how

the mid moved after each individual fill, and they are computed at different grains. They share the same fill inference, so agreement does not verify that inference; what it does show is that the price a window paid is consistent with what the market did immediately afterwards. A discrepancy would indicate that the simulator is not pricing executions correctly, and there is none.

Figure 1 compares the mark-out of the two execution styles, Figure 2 their relative slippage, and Figure 3 puts the two measures side by side for both.

Table 4: Three measures of the same executions, in ticks per contract. *Relative slippage* prices the two legs of a window against the arrival mid and is a per-window quantity. *Mark-out* is the mid-price move over the stated horizon after each individual fill, one row per execution, signed so that positive is a move against the fill. Both are priced against the mid, so they are on the same scale and directly comparable. They measure *related* quantities rather than one quantity twice: relative slippage is the price a completed round trip achieved, mark-out is how the mid moved after each individual fill. Agreement between them is therefore a consistency check — what a window paid lines up with what happened after its fills — and not a replication. Both rows cover the full corpus: 245 sessions and 1.63 million individual fills for the passive style, 246 sessions and 1.64 million for the aggressive. Intervals are 95 % and clustered by session.

	Rel. slippage	Mark-out after the fill		
	(vs. mid)	1 s	5 s	30 s
SHADOW-PPOV (passive)	$+0.0953 \pm 0.0130$	$+0.0825 \pm 0.0076$	$+0.0743 \pm 0.0110$	$+0.0993 \pm 0.0200$
Aggressive POV	$+0.0952 \pm 0.0135$	$+0.0826 \pm 0.0072$	$+0.0874 \pm 0.0118$	$+0.0913 \pm 0.0190$

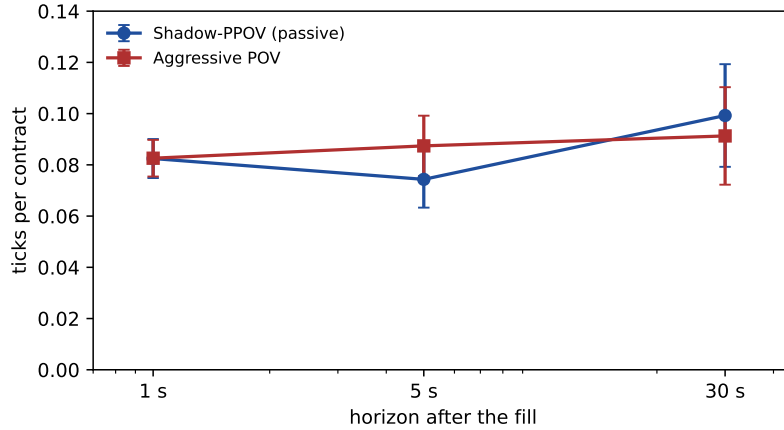


Figure 1: Post-fill mark-out for the two execution styles, at one, five and thirty seconds, with 95 % intervals clustered by session. The two are indistinguishable at every horizon, and each profile is flat rather than decaying: the price move is complete within a second of the fill and does not revert over the following twenty-nine. That is the signature of adverse selection rather than of temporary impact.

The mark-outs agree with the window-level costs and with each other. Both styles give up something under a tenth of a tick per contract to the subsequent mid-price, and the two are indistinguishable at every horizon. The profile is flat in time rather than decaying: the price move is complete within a second of the fill and does not revert over the following twenty-nine. That is the signature of adverse selection rather than of temporary impact — the fill happened because someone with a reason to trade took the other side, and the reason does not go away.

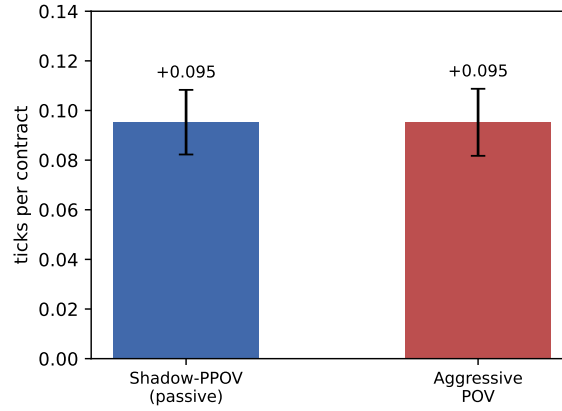


Figure 2: Window-level relative slippage for the two execution styles at matched participation, with 95 % intervals clustered by session. One style rests for 99 % of its executed quantity and the other crosses for half of it; the round trip costs the same either way.

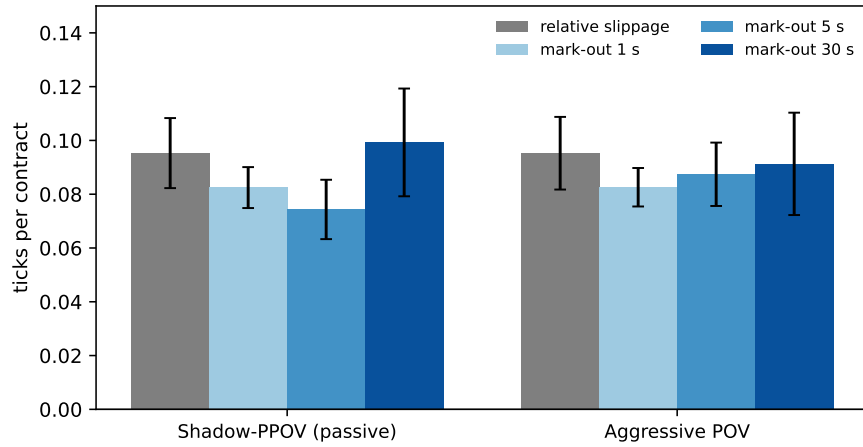


Figure 3: Relative slippage against mark-out, both execution styles. The two quantities are computed from different data at different grains — one from the volume-weighted prices of a whole window, the other from the mid-price after each individual fill — so their agreement is a check on the measurement rather than an identity. They agree to within a couple of hundredths of a tick throughout.

8.4 Latency

Of everything varied in this study, the round-trip delay is the parameter that moves the cost. It moves it monotonically, in both execution styles, with non-overlapping confidence intervals from end to end.

The delay is applied to *all three* paths at once — the outbound order, the outbound cancel, and the inbound market-data feed — because a study that delays only the outbound side measures an algorithm that still sees the book instantly, which no participant does. Table 5 reports both styles across the range.

1. Cost rises monotonically. Passive SHADOW-PPOV costs +0.097 ticks per contract at zero delay and +0.150 at 5 ms, a 55 % increase; aggressive POV goes from +0.087 to +0.296, more than a tripling. Every step in both columns lies outside the preceding cell’s interval, and Figure 4 plots them. How fast the algorithm reaches the exchange governs what it pays.

The two do not degrade at the same rate, and the difference is the more interesting half of the table. Averaged over the range, passive SHADOW-PPOV loses 0.011 ticks per contract per millisecond of delay and aggressive POV loses 0.042 — roughly four times as fast. The ordering reverses inside the first half-millisecond: at zero delay aggressive POV is the cheaper of the two by 0.010 ticks, and by 500 μ s it is the more expensive by 0.055. Measured against immediate execution the asymmetry is starker still. Passive SHADOW-PPOV gives up 0.051 of its 0.422 advantage across the whole range, about an eighth of it; aggressive POV gives up 0.207 of 0.432, very nearly half. Everything concluded in Section 8.3 from the zero-latency comparison — that the two styles cost the same, and that the choice between them is one about speed rather than price — holds at zero latency and nowhere else in this table.

2. Neither style holds its character. The mechanism is the share of executed quantity that fills on arrival rather than resting. A passive quote is priced against a book that has moved by the time the order lands, so it arrives marketable: passive SHADOW-PPOV goes from 1.1 % of its quantity filling on arrival to 27.3 %, a quarter of it converted into liquidity taking by latency alone, with no change to its logic. Aggressive POV drifts the other way at first — at 500 μ s its marketable limits arrive after the level they were priced from has been consumed, and rest instead — and then upward, to 74.6 %, as the market moves far enough during the flight that the limit is through the touch on arrival. The two styles converge toward each other.

3. Participation moves in opposite directions. Figure 5 is the one place in this study where the two styles do not merely differ in degree. Passive participation *rises* with delay, 4.51 % $\rightarrow$ 6.87 %, half again as much of the market’s volume; aggressive participation *falls*, 3.18 % $\rightarrow$ 2.02 %, to under two thirds of what it was.

4. The degradation is in the price, not only in the mix. Relative slippage rises monotonically in both styles while the quantity executed and the number of child fills are unchanged, so the algorithm is paying more for the same executions rather than doing a different amount of business. How much of that is the style conversion of item 2 and how much is worse execution within a style cannot be separated from these runs alone; what can be said is that both channels push the same way and neither reverses anywhere in the range.

5. The latency sensitivity is itself evidence that the method extracts information from the flow. A policy whose actions are independent of order flow has no timing relationship with

Table 5: Latency sensitivity, the same delay on order, cancel and feed. ES futures, 246 sessions of 2025, 100 contracts bought and 100 sold per window, every other parameter fixed within each style. The configurations here are *not* the pair of Table 3: that pair was selected for the closest realized participation match, whereas these two were each left at their own working settings so that the delay is the only thing varied within a style. Both styles therefore sit at a different placement rate here than in Table 3: the aggressive one shadows 1 % of prints against 2 % there, which is why its zero-delay row reads 3.18 % participation and 92s rather than 4.86 % and 62.4s, and the passive one reads 4.51 % against 4.46 %. Rows are comparable down a column; across tables they are not.

Delay (μ s)	Windows	Rel. slippage (ticks/contract)	vs. immediate (ticks/contract)	Participation (%)	Time (s)
<i>Passive SHADOW-PPOV</i>					
0	9,591	$+0.097 \pm 0.013$	-0.422	4.51	72
500	9,593	$+0.121 \pm 0.014$	-0.400	6.00	55
1000	9,592	$+0.132 \pm 0.015$	-0.388	6.12	54
2500	9,593	$+0.141 \pm 0.015$	-0.380	6.40	52
5000	9,594	$+0.150 \pm 0.015$	-0.371	6.87	48
<i>Aggressive POV</i>					
0	9,584	$+0.087 \pm 0.017$	-0.432	3.18	92
500	9,541	$+0.176 \pm 0.017$	-0.345	2.43	121
1000	9,525	$+0.211 \pm 0.018$	-0.309	2.24	131
2500	9,531	$+0.240 \pm 0.018$	-0.280	2.09	138
5000	9,564	$+0.296 \pm 0.022$	-0.225	2.02	141

that flow, and therefore nothing for delay to degrade: place at arbitrary times and prices and it should not much matter whether the order reaches the exchange now or five milliseconds from now. What Table 5 shows is the opposite. Cost rises monotonically in both styles, with non-overlapping intervals from end to end, so *when* an order arrives relative to the flow it is following is worth 55 % of the passive cost and more than a tripling of the aggressive one. That is only possible if the placement and cancellation decisions stand in a real temporal relationship to the order flow — which is to say that following an identified order carries information, rather than amounting to posting at a sensible depth by another route.

The slope measures how tightly a method is coupled to the flow, and the two styles rank as the mechanism predicts. Aggressive POV must *reach* a price the flow has just printed, the tightest coupling available, and loses 0.042 ticks per contract per millisecond. Passive SHADOW-PPOV inherits a price and a lifetime but waits to be reached, and loses 0.011.

8.5 A control: the same method on stale information

The latency sweep above measures a slope over delays a real participant might actually face. This subsection is a different kind of experiment and is kept separate for that reason: thirty seconds is not an operating point, it is a *control*. The question it answers is how much of SHADOW-PPOV’s cost advantage comes from the information it inherits, and it answers it by destroying that information’s freshness while holding everything else fixed.

Cost rises by a factor of 3.9, on intervals that do not approach each other, and three further columns move with it: participation falls to a third, the parent is worked in 37 child fills per leg rather than 85, and the legs finish *sooner* — because a quote priced thirty seconds ago arrives marketable and crosses on contact rather than resting, which is item 2 of Section 8.4 at an extreme. The difference, 0.272 ticks per contract, is what acting immediately on an observed order is worth

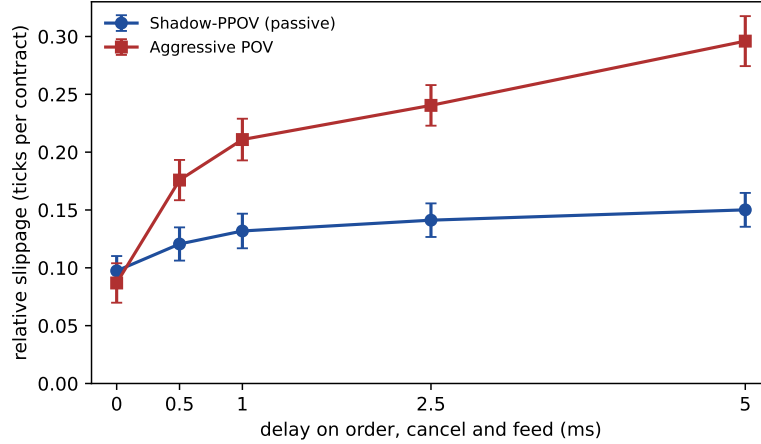


Figure 4: Relative slippage against round-trip delay — the cost column of Table 5 — with 95 % intervals clustered by session. The same delay is applied to the outbound order, the outbound cancel and the inbound feed. The two styles start together and separate at once: aggressive POV is the cheaper of the two at zero delay and the more expensive by 500 μ s, and its slope over the range is about four times the passive one.

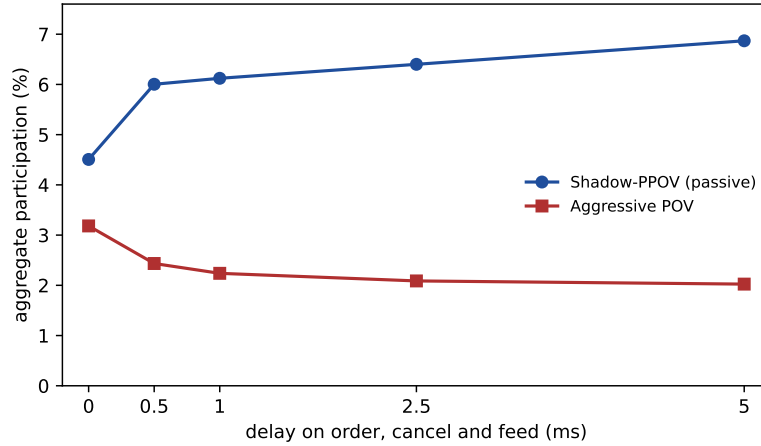


Figure 5: Aggregate participation against round-trip delay, the same runs as Figure 4. Passive SHADOW-PPOV takes a *larger* share of the market’s volume as it gets slower and aggressive POV a smaller one. Both follow from the conversion of execution style: the passive side turns quotes into crosses and finishes sooner, the aggressive side loses crosses and takes longer, and participation is scored over the leg’s own duration.

Table 6: The same passive configuration run against itself, with and without a thirty-second delay on the order, the cancel and the feed. ES futures, 245 sessions of calendar 2025 without the delay and 246 with it, 100 contracts bought and 100 sold per window, the passive configuration of Table 3 throughout. Thirty seconds is chosen as a control rather than as a plausible latency: it is long enough that the observation the shadow was placed from has been overtaken by the market many times over. Intervals are 95 % and clustered by session.

Delay	Windows	Rel. slippage (ticks/contract)	Participation (%)	Time (s)	Child fills (per leg)
None	9,554	$+0.0953 \pm 0.0130$	4.47	72	85.5
30 seconds	9,594	$+0.3673 \pm 0.0989$	1.56	44	37.2

over acting on the same observation half a minute later.

What this establishes is that the inherited information is real and that it is *perishable*. It does not establish what that information is worth against placing with none at all — at thirty seconds the price is still inherited from a real participant’s order, merely stale.

The zero-delay row also reproduces the passive column of Table 3 on an independent re-run: +0.0953 against +0.0955, 4.47 % participation against 4.46 %, and 72s against 72.7s.

8.6 Threats to validity

The material limitation is that the replay does not model the market’s reaction to SHADOW-PPOV’s own orders. The simulator fills the algorithm’s orders against the recorded feed as though those orders had not been there: a fill is *added* to the day’s volume rather than displacing someone else’s. At the 4.5 % aggregate participation of the zero-latency comparison that assumption is mild. At the 6.9 % reached by passive SHADOW-PPOV at 5 ms it is strained — and individual windows reach far higher — so the true denominator would be smaller and the reported participation should be read as an upper bound.

Two further caveats. The study is a single instrument: ES is a one-tick book for most of the session, and the relative-to-immediate figure in particular is close to an identity in such a book, so it should not be carried over to wider-spread products without re-measurement. And the parent size is fixed at 100 contracts throughout; the size dependence is the subject of the next section.

9 Participation Rate and Parent-Order Size

SHADOW-PPOV controls its *order-placement rate* (Section 6.3); it does not control the participation rate directly, nor the fill of any one shadow. Because each shadow joins the queue behind the add it attaches to, while the amount of depth ahead of it is unknown at placement, whether and when it fills is a random event. A small parent has few fill opportunities, so its realized participation is highly variable; as the number of completed child fills grows, averaging reduces that variability. This is a general property of passive percentage-of-volume and volume-tracking execution rather than a feature specific to SHADOW-PPOV: the delivered participation is a realized quantity, a recognized source of tracking error and execution risk [6, 7, 8, 9]. SHADOW-PPOV inherits this behavior, with the queue-position uncertainty of the followed order as the concrete source of per-fill randomness [23, 19]; the dependence of that dispersion on parent-order size is what the model below quantifies.

A stylized model. To isolate the dependence on size, consider executing a parent of Q one-lot children. Each child attaches to a random add whose queue position is unknown. For illustration we assume the market volume that must trade before a child fills is $\text{Exponential}(\rho)$ with mean $1/\rho$ for a target participation rate ρ (within this stylized model ρ denotes the participation rate, distinct from the order-placement rate of Section 6.3). The market volume elapsed when the parent completes its Q fills is

$$M = \sum_{i=1}^Q G_i, \quad G_i \sim \text{Exponential}(\rho) \implies M \sim \text{Gamma}(Q, \rho),$$

and the realized participation is $\hat{p} = Q/M$. The ratio $\hat{p}/\rho = Q/\text{Gamma}(Q, 1)$ is independent of ρ , so the dependence on ρ cancels and the model predicts the same normalized size dependence for any ρ . The qualitative conclusion — concentration on the model’s mean as Q grows — follows from the central limit theorem and holds for a broad class of finite-variance gap distributions. The independent-exponential gap assumption is a strong simplification: real fill gaps are correlated across

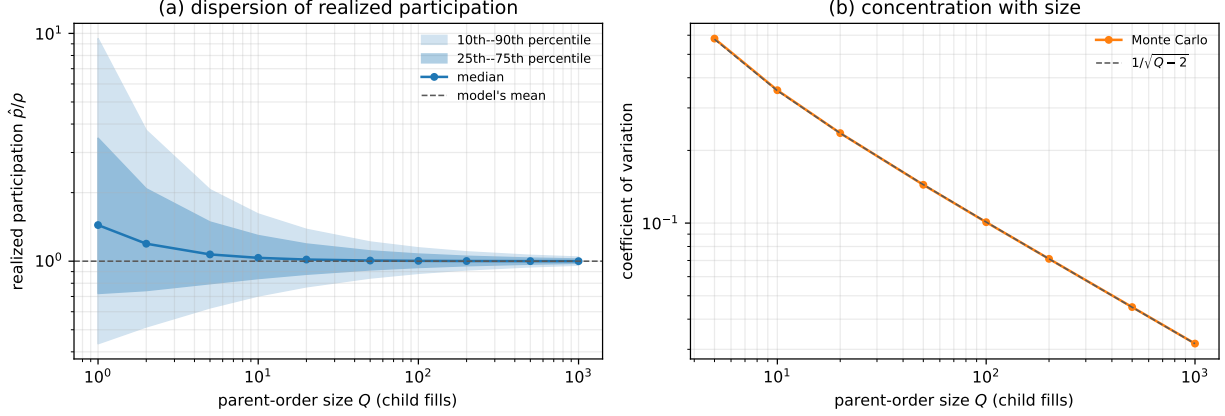


Figure 6: Realized participation relative to the model’s mean versus parent-order size Q (number of one-lot child fills), from the stylized model $\hat{p}/\rho = Q/\text{Gamma}(Q, 1)$. (a) Median and percentile bands: the ratio is dispersed for small Q and concentrates on the model’s mean (dashed line) as Q grows. (b) The coefficient of variation falls as $1/\sqrt{Q-2}$.

Table 7: Participation relative to the model’s mean versus parent-order size, 4×10^5 Monte Carlo trials per size. CV is finite only for $Q > 2$.

Q (child fills)	median	10th pct	90th pct	CV
1	1.44	0.44	9.48	—
2	1.19	0.51	3.77	—
5	1.07	0.63	2.06	0.58
10	1.03	0.70	1.61	0.35
20	1.02	0.77	1.38	0.24
50	1.01	0.84	1.22	0.14
100	1.00	0.89	1.14	0.10
200	1.00	0.92	1.10	0.07
500	1.00	0.95	1.06	0.05
1000	1.00	0.96	1.04	0.03

concurrent shadows, volatility regimes, and clustered aggressive events, and modeling the children as filling one at a time produces greater participation variance than allowing multiple concurrent fills to average independently. The model does not estimate SHADOW-PPOV’s actual participation — the exponential fill-gap distribution and its rate are not fitted to the CME corpus — so the result is a scaling law under a stylized model, not an empirical claim that SHADOW-PPOV achieves a particular participation rate.

Results. Figure 6 and Table 7 report a Monte Carlo of $\hat{p}/\rho$ over 4×10^5 trials per size. At $Q = 1$ the ratio is $1/\text{Exponential}(1)$: heavy-tailed with a divergent mean and a 10th–90th percentile band spanning $[0.44, 9.5]$ times the mean — the participation is effectively random. As Q grows the ratio concentrates on the model’s mean and its coefficient of variation falls as $1/\sqrt{Q-2}$ (asymptotically $1/\sqrt{Q}$): the 10th–90th band narrows to $[0.70, 1.61]$ at $Q = 10$, $[0.89, 1.14]$ at $Q = 100$, and $[0.96, 1.04]$ at $Q = 1000$.

Implication. The size at which realized participation becomes concentrated bounds where SHADOW-PPOV, or any passive follower, delivers a stable participation. Under the stylized

model a parent comprising hundreds or thousands of child fills has realized participation within a few percent of the model’s mean; a very small order does not, and its participation is effectively random. For a small parent order the dispersion can be reduced by modifying SHADOW-PPOV to shadow only adds reasonably close to the inside of the book (tightening the placement band of Section 7), which have a higher and more predictable fill probability; without such a restriction a small order’s participation is dominated by the queue-position randomness of the few orders it follows, as the small- Q case shows. This also bounds the proposal of Section 10: the benchmark role is proposed for parent sizes large enough that the participation band is narrow, and the width of that band is a second axis — alongside slippage — on which a predictive placement model that controls queue position could be scored against it.

10 A benchmark for passive placement

SHADOW-PPOV places passive orders without an order-book model and without predicting where they will fill, and it executes at the relative slippage measured in Section 8. We propose it as a benchmark for passive-order placement on a CLOB rather than as an optimal method.

Why a model-free passive method should not beat the mid-price. The sign of the measured cost is not an accident of this implementation, and it is worth stating as a prediction rather than reporting as an outcome. In the Glosten–Milgrom account of the spread [14], the quoted width exists to compensate the party supplying liquidity for trading against possibly better-informed flow: a resting order is filled precisely when somebody wants to take the other side, so the half-spread it captures is paid back, in expectation, through adverse selection. A passive method carrying *no* forecast of where the price is going has nothing with which to separate the fills it wants from the fills it does not, and should therefore execute slightly worse than the mid-price — which is what Section 8 measures.

Two consequences follow, and they are what make the method useful as a reference. First, a passive result at or better than mid from a model-free method would indicate an error in the measurement or an unmodelled edge, so the small positive cost is a sanity condition the benchmark must satisfy, not a weakness to be minimized. Second, the same argument predicts that the expected *cost* of resting and of crossing should converge, leaving the choice between them a choice about speed and participation rather than about price. A predictive placement model is then scored on exactly the quantity SHADOW-PPOV cannot address: how much of that adverse-selection cost it recovers by declining the fills a forecast identifies as toxic. Given the same passive mandate, a predictive placement model can be scored against SHADOW-PPOV to test whether its additional modeling produces a measurable improvement; the present study reports no such competing baselines, so the benchmark role is proposed rather than demonstrated.

Adaptive scheduling methods are already evaluated against fixed references: a reinforcement-learning or optimal-execution scheduler is judged against TWAP, VWAP, and the Almgren–Chriss trajectory [1, 28], so any claimed gain is stated relative to a schedule that carries no forecast of its own. That works because a schedule crossing every slice is fully specified by its clock. A TWAP that works its slices passively is not: it still has to decide where in the book each slice rests, and that decision needs the very placement model the reference was supposed to be free of (Section 2). So the fixed reference the literature already has covers the schedule, and stops at the point the order goes passive.

That is the gap. The naive alternatives at the placement layer — posting at the touch, or post-and-wait then cross [18, 21] — are ad hoc, and each smuggles in a choice of depth or deadline

that the next author will make differently. SHADOW-PPOV is well specified in a page, has no free parameter that decides where to rest, and can be re-run by anyone with the same data. A practitioner proposing a predictive placement model can then show whether it beats shadow-following, as an execution scheduler shows whether it beats TWAP.

The bar a placement model has to clear. Stated plainly, the proposal is a floor and it has two rungs. SHADOW-PPOV places passively with no view at all, and aggressive POV crosses with no view at all; between them they bracket what is achievable by following flow and nothing else. A designer proposing a predictive placement model should have to beat *both* on the same corpus, the same parent, and the same participation: beating SHADOW-PPOV shows the forecast is worth something over resting blind, and beating aggressive POV shows it is worth something over simply taking liquidity. A model that beats neither has bought complexity and a fitting risk in exchange for nothing, and its author should be able to see that from two numbers. This is the role we propose — not a claim that SHADOW-PPOV is hard to beat, but a claim that it is the right thing to be measured against, in the way a scheduler is measured against TWAP.

SHADOW-PPOV is an execution method: its scope is completing a given position at a small, predictable cost. When direction is known, an informed aggressive strategy outperforms it, and SHADOW-PPOV leaves such directional decisions to the strategy layer above it.

11 Future Work

The measurements of Section 8 establish the mechanism and characterize its cost over one instrument and one year. A completed benchmark study needs more than that, and several directions remain before SHADOW-PPOV can be treated as a validated reference.

Comparison against established methods. The central open task is to score SHADOW-PPOV against competing passive-placement rules under the same direction-neutral slippage metric: posting at the touch; the passive legs of TWAP, VWAP, and POV; and at least one *predictive* placement model. Only such a comparison can establish whether SHADOW-PPOV is a useful benchmark and by how much a predictive model improves on it. Section 8.5 establishes that the inherited information has value and measures how quickly it decays; what a comparison adds is where the method sits against the alternatives a desk would otherwise reach for.

Understanding the algorithm’s behavior. Beyond a single cost figure, the method’s behavior needs characterization: realized fill and completion rates, the fraction of followed orders whose lifecycle ends in a fill rather than a cancellation, queue-position dynamics, and sensitivity to the order-placement rate ρ and the grace window B .

Ablating the guards. The safety guards (the two-band hysteresis and the completion and over-hedge limits) should be ablated to quantify their effect on the reported costs.

Market impact and breadth. The replay uses a zero-impact simulator; a controlled experiment that injects synthetic size and measures the perturbation of subsequent flow would test that assumption. The corpus is a single index-futures contract; extending to other products, to fragmented multi-venue equity markets — where the flow-inherited venue selection of Section 6.7 would actually be exercised — and to larger sizes is required before the results can be generalized.

12 Conclusion

We presented SHADOW-PPOV, a passive execution method that trades the schedules and aggregate-volume targets of VWAP/TWAP/POV for direct observation of the passive orders participants actually post. By shadowing individual resting orders by exchange identifier — mirroring price and venue, and tying cancellation to the shadowed order’s lifecycle through a lookup on the followed order’s identifier — SHADOW-PPOV works without a volume forecast of its own, uses followed orders as observable references for local execution flow, and obtains venue selection as an emergent property of the flow it follows rather than from a routing computation. Over a full calendar year of replayed CME ES sessions we compared passive SHADOW-PPOV against an aggressive equivalent at nearly matched participation, and swept the simulated latency from zero to five milliseconds on the order, the cancel and the feed together.

Three results stand out. *The method executes close to the mid.* Over 9,592 completed passive windows a round trip of 100 contracts bought and 100 sold costs $+0.0955 \pm 0.0130$ ticks per contract against the arrival mid — under a tenth of a tick, or \$1.19 per contract on ES — and beats immediate execution (crossing the spread) by 0.4236 ticks. Post-fill mark-outs computed from different data at a different grain agree with that figure, so it is a measurement rather than an artifact of one definition.

Passive and aggressive cost the same, at zero latency. At matched participation the aggressive equivalent returns $+0.0952 \pm 0.0135$ against passive’s $+0.0955$, and -0.4239 against -0.4236 relative to immediate execution — agreement to the fourth decimal on both, over more than nine thousand windows each. This is what the Glosten–Milgrom account of the spread predicts of a method carrying no forecast: the half-spread a resting order captures is returned through adverse selection, and the half-spread a crossing order pays buys the avoidance of it.

Latency degrades both, and the method remains usable throughout the range tested. Cost rises monotonically with round-trip delay in both styles, with non-overlapping intervals from end to end. Passive SHADOW-PPOV goes from $+0.097$ to $+0.150$ ticks per contract between zero and five milliseconds, still beating immediate execution by 0.371 ticks at the far end; aggressive POV more than triples over the same range, degrading about four times as fast, and the ordering between the two reverses inside the first half-millisecond.

Taken together these say the two styles are complementary rather than competitive. They are indistinguishable on price when the algorithm is fast, so the choice between them is a choice about speed, participation and completion — and under delay they fail in opposite directions, passive gaining participation as its quotes convert into crosses and aggressive losing it as its marketable orders miss. An implementation that can reach the exchange quickly has a genuine choice; one that cannot should rest.

Because its placement decision reads a price off an observed order and stops there, we propose SHADOW-PPOV as a model-free benchmark for passive-order placement — a reference a predictive placement model should be expected to beat, on the improvement it delivers over these numbers and at a latency it can actually achieve.

Reproducibility and disclosure

Results are produced by a deterministic replay of recorded CME PCAP market data through open-source C++ actor code (Section 7.1); a fixed session replays to identical fills. The SHADOW-PPOV implementation and the order-book simulator are available as the KASPAR-HFT project.²

²<https://github.com/vincent212/kaspar-hft>

Acknowledgments. The slippage-measurement corpus and the deterministic replay harness were built at M2 Technologies.

References

- [1] R. Almgren and N. Chriss. *Optimal Execution of Portfolio Transactions*. Journal of Risk, 3(2):5–40, 2001.
- [2] M. Avellaneda and S. Stoikov. *High-Frequency Trading in a Limit Order Book*. Quantitative Finance, 8(3):217–224, 2008.
- [3] C. Hewitt, P. Bishop, and R. Steiger. *A Universal Modular ACTOR Formalism for Artificial Intelligence*. Proc. 3rd International Joint Conference on Artificial Intelligence (IJCAI), 1973.
- [4] G. Agha. *Actors: A Model of Concurrent Computation in Distributed Systems*. MIT Press, 1986.
- [5] L. Harris. *Trading and Exchanges: Market Microstructure for Practitioners*. Oxford University Press, 2003.
- [6] R. Kissell. *The Science of Algorithmic Trading and Portfolio Management*. Academic Press, 2013.
- [7] Á. Cartea, S. Jaimungal, and J. Penalva. *Algorithmic and High-Frequency Trading*. Cambridge University Press, 2015.
- [8] J. Białkowski, S. Darolles, and G. Le Fol. *Improving VWAP Strategies: A Dynamic Volume Approach*. Journal of Banking & Finance, 32(9):1709–1722, 2008.
- [9] C. Frei and N. Westray. *Optimal Execution of a VWAP Order: A Stochastic Control Approach*. Mathematical Finance, 25(3):612–639, 2015.
- [10] B. Johnson. *Algorithmic Trading & DMA: An Introduction to Direct Access Trading Strategies*. 4Myeloma Press, London, 2010.
- [11] V. Markov. *Bayesian Trading Cost Analysis and Ranking of Broker Algorithms*. [arXiv:1904.01566](https://arxiv.org/abs/1904.01566), 2019.
- [12] A. F. Perold. *The Implementation Shortfall: Paper versus Reality*. Journal of Portfolio Management, 14(3):4–9, 1988.
- [13] Á. Cartea and S. Jaimungal. *Optimal Execution with Limit and Market Orders*. Quantitative Finance, 15(8):1279–1291, 2015.
- [14] L. R. Glosten and P. R. Milgrom. *Bid, Ask and Transaction Prices in a Specialist Market with Heterogeneously Informed Traders*. Journal of Financial Economics, 14(1):71–100, 1985.
- [15] O. Guéant, C.-A. Lehalle, and J. Fernández-Tapia. *Optimal Portfolio Liquidation with Limit Orders*. SIAM Journal on Financial Mathematics, 3(1):740–764, 2012. [arXiv:1106.3279](https://arxiv.org/abs/1106.3279).
- [16] O. Guéant. *Execution and Block Trade Pricing with Optimal Constant Rate of Participation*. 2012. [arXiv:1210.7608](https://arxiv.org/abs/1210.7608).

- [17] J. Vaes and R. Hauser. *Optimal Trade Execution with Uncertain Volume Target*. Journal of Computational Finance, 2022. [arXiv:1810.11454](#).
- [18] R. Cont and A. Kukanov. *Optimal Order Placement in Limit Order Markets*. Quantitative Finance, 17(1):21–39, 2017. [arXiv:1210.1625](#).
- [19] T. DeLise. *The Negative Drift of a Limit Order Fill*. [arXiv:2407.16527](#), 2024.
- [20] V. Markov, S. Mazur, and D. Saltz. *Design and Implementation of Schedule-Based Trading Strategies Based on Uncertainty Bands*. The Journal of Trading, Fall 2011. [arXiv:1409.1441](#).
- [21] V. Markov. *On the Design of Sell-Side Limit and Market Order Tactics*. The Journal of Trading, 7(3):29–39, 2012. [arXiv:1409.1442](#).
- [22] R. Cesari, M. Marzo, and P. Zagaglia. *Effective Trade Execution*. [arXiv:1206.5324](#), 2012.
- [23] C. C. Moallemi and K. Yuan. *A Model for Queue Position Valuation in a Limit Order Book*. Working paper, Columbia University, 2016. SSRN 2996221.
- [24] T. Foucault and A. J. Menkveld. *Competition for Order Flow and Smart Order Routing Systems*. Journal of Finance, 63(1):119–158, 2008. [doi:10.1111/j.1540-6261.2008.01312.x](#).
- [25] R. Battalio, S. A. Corwin, and R. Jennings. *Can Brokers Have It All? On the Relation between Make-Take Fees and Limit Order Execution Quality*. Journal of Finance, 71(5):2193–2238, 2016. [doi:10.1111/jofi.12422](#).
- [26] S. Laruelle, C.-A. Lehalle, and G. Pagès. *Optimal Split of Orders Across Liquidity Pools: A Stochastic Algorithm Approach*. SIAM Journal on Financial Mathematics, 2(1):1042–1076, 2011. [arXiv:0910.1166](#).
- [27] A. Barzykin, R. Boyce, E. Neuman, and S. Tuschmann. *Optimal Execution with Passive Market Impact*. [arXiv:2607.28323](#), 2026.
- [28] T. Espana, Y. Hafsi, F. Lillo, and E. Vittori. *Reinforcement Learning in Queue-Reactive Models: Application to Optimal Execution*. [arXiv:2511.15262](#), 2025.
- [29] P. Cheridito and M. Weiss. *Reinforcement Learning for Trade Execution with Market and Limit Orders*. [arXiv:2507.06345](#), 2025.
- [30] J. Albers, M. Cucuringu, S. Howison, and A. Y. Shestopaloff. *The Market Maker’s Dilemma: Navigating the Fill Probability versus Post-Fill Returns Trade-Off*. [arXiv:2502.18625](#), 2025.
- [31] Deltix Ember. *PVOL Algorithm — product documentation*. [ember.deltixlab.com/docs/algos/pvol](#). Accessed September 2026.
- [32] TradeStation. *POV-ALGO — routing documentation*. [help.tradestation.com](#). Accessed September 2026.
- [33] *Tracking Liquidity Order*. U.S. Patent No. 10,614,520 B2, 2020. [patents.google.com/patent/US10614520B2](#).
- [34] *Copy Trading System and Method*. U.S. Patent Application Publication No. 2013/0268423 A1 (Pennaluna & Company, Inc.), 2013. [patents.google.com/patent/US20130268423A1](#).

- [35] *Passive Liquidity Order*. U.S. Patent Application Publication No. 2006/0253379 A1, 2006.
- [36] *Algorithmic Trading — Hidden Liquidity*. U.S. Patent No. 8,140,416 B2, 2012.